# HeartGuard AI — Complete Project & Code Walkthrough

**AI-Powered Heart Disease Risk Assessment, Explainable AI & Lifestyle Intelligence**


**Author / Presenter:** Praneeth Badugu

**Domain:** Healthcare AI / Deep Learning

**Stack:** PyTorch, FastAPI, React 18, TypeScript, Tailwind

**Repository:** github.com/praneethbadugu7781-create/heartai

**Live Demo Web:** heartai-three.vercel.app

**Live API Docs:** heartai-kyzq.onrender.com/docs


## 1. Executive Summary & Problem Statement

Cardiovascular diseases (CVDs) are the leading cause of death globally, taking an estimated 17.9 million lives each year. Conventional diagnostic methods (angiography, nuclear perfusion scans, stress tests) are invasive, costly, and delayed. **HeartGuard AI** is a production-grade preventive healthcare platform that leverages multiple machine learning models (Deep Neural Network, Multi-Layer Perceptron, and TabNet Attention) to estimate cardiovascular risk from non-invasive biometrics and provide transparent **Explainable AI (XAI)** feature attributions and **6-Pillar Lifestyle Intelligence**.

**Crucial Medical Boundary:** The platform operates strictly as an *educational and preventive risk estimation tool*, not an automated diagnostic system. It includes automated emergency symptom escalation (detecting acute chest tightness, radiating pain, fainting) directing users to emergency services immediately.

## 2. Clinical Dataset & Zero-Leakage Preprocessing

The models are trained on the benchmark **UCI Cleveland / Kaggle Heart Disease Dataset** comprising 1025 patient records with 14 clinical features:

| Feature Code | Clinical Description | Normal / Reference Range | Type |
|---|---|---|---|
| age | Patient age in years | 18 - 105 yrs | Numerical |
| sex | Biological sex | 1 = Male, 0 = Female | Binary |
| cp | Chest pain type | 0: Typical, 1: Atypical, 2: Non-anginal, 3: None | Categorical |
| trestbps | Resting systolic blood pressure | Normal: <120, Stage 1: 130-139, Stage 2: ≥140 | Numerical (mmHg) |
| chol | Serum total cholesterol | Desirable: <200, Borderline: 200-239, High: ≥240 | Numerical (mg/dL) |
| fbs | Fasting blood sugar > 120 mg/dL | 1 = True (elevated), 0 = False (normal) | Binary |
| restecg | Resting electrocardiographic results | 0: Normal, 1: ST-T wave wave abnormality, 2: LVH | Categorical |
| thalach | Maximum heart rate achieved | Target max: (220 - age) bpm | Numerical (bpm) |

| | | | |
|---|---|---|---|
| exang | Exercise-induced angina | 1 = Yes, 0 = No | Binary |
| oldpeak | ST depression induced by exercise | 0.0 mm (normal) to >2.0 mm (ischemia) | Numerical (mm) |
| slope | Slope of peak exercise ST segment | 0: Upsloping, 1: Flat, 2: Downsloping | Categorical |
| ca | Major coronary vessels colored | 0 to 3 vessels (0 = clear) | Numerical |
| thal | Thalassemia / Nuclear perfusion scan | 1: Normal, 2: Fixed defect, 3: Reversible defect | Categorical |
| target | Cardiovascular risk classification | 0 = Lower estimated risk, 1 = Higher estimated risk | Target Label |

**Zero Data Leakage Methodology:** The data is partitioned into a Stratified 70% Train (717 samples), 15% Validation (154 samples), and 15% Independent Test (154 samples). Preprocessor scalers (StandardScaler) and median imputers are **strictly fitted on the 70% training split only**, ensuring the test evaluation is completely unbiased and representative of real-world clinical generalization.

# 3. Multi-Model Deep Learning Architectures

Rather than relying on a single algorithm with inherent inductive biases, HeartGuard AI trains three fundamentally different paradigms:

### A. Deep Neural Network (DNN) — Non-Linear Feature Combinations

• **Architecture:** Input(13) → Dense(64) → BatchNorm1d → ReLU → Dropout(0.25) → Dense(32) → BatchNorm1d → ReLU → Dropout(0.20) → Dense(16) → ReLU → Dense(1) → Sigmoid.
• **Purpose:** Captures complex non-linear clinical interactions (e.g. resting BP combined with ST depression and elevated cholesterol). Batch Normalization prevents internal covariate shift.

### B. Multi-Layer Perceptron (MLP) — Compact Regularized Baseline

• **Architecture:** Input(13) → Dense(32) → LeakyReLU(0.1) → Dropout(0.15) → Dense(16) → LeakyReLU(0.1) → Dense(1) → Sigmoid.
• **Purpose:** Provides a highly regularized, low-complexity parametric baseline with L2 weight decay to prevent overfitting on smaller clinical cohorts.

### C. TabNet Classifier — Sequential Attention Transformer

• **Architecture:** 3-Step Sequential Attentive Transformer with Gated Linear Unit (GLU) feature transformers and Sparsemax/Entmax attention masks.
• **Purpose:** Uses sparse instance-wise feature selection at each decision step, mirroring human clinical diagnostic reasoning by selecting the most critical features first.

### D. Calibrated Ensemble Formulation

The final risk score combines all three models via a performance-weighted consensus formula:

**P_ensemble = 0.35 · P_DNN + 0.30 · P_MLP + 0.35 · P_TabNet**

• **Risk Categories:** <35% = Lower Estimated Risk, 35%–65% = Moderate Estimated Risk, >65% = Higher Estimated Risk.

# 4. Empirical Model Performance (Held-Out Test Set)

All metrics below were computed on the independent 15% test cohort (154 samples) with zero data leakage:

| Model Architecture | Accuracy | Precision | Recall (Sensitivity) | Specificity | F1-Score | ROC-AUC |
|---|---|---|---|---|---|---|
| Deep Neural Network (DNN) | 85.7% | 89.5% | 82.9% | 88.9% | 86.1% | 0.936 |
| Multi-Layer Perceptron (MLP) | 83.1% | 85.9% | 81.7% | 84.7% | 83.8% | 0.922 |
| TabNet Attention Model | 83.8% | 89.0% | 79.3% | 88.9% | 83.9% | 0.916 |
| HeartGuard Calibrated Ensemble | 84.4% | 89.2% | 80.5% | 88.9% | 84.6% | 0.934 |

## 5. Explainable AI (XAI) & 6 Lifestyle Intelligence Pillars

**Explainable AI (XAI):** The platform extracts TabNet attention mask coefficients and neural gradient saliencies to quantify how much each physiological marker contributed to the prediction. It classifies factors into **Risk Elevators** (e.g. Stage-2 BP ≥ 140, Exercise ST depression > 1.5mm) and **Protective Factors** (e.g. Normal baseline ECG, asymptomatic presentation).

**6-Pillar Lifestyle Intelligence:** Translates complex neural outputs into actionable preventive guidance:
1. **Nutrition:** Sodium restriction (<2000 mg/day for elevated BP), DASH / Mediterranean dietary patterns.
2. **Physical Activity:** 150 mins/week moderate aerobic activity tailored to exercise-induced angina status.
3. **Sleep Architecture:** 7–9 hours consistent sleep to support autonomic recovery and cortisol regulation.
4. **Stress Management:** HRV coherence, vagal stimulation, progressive muscle relaxation.
5. **Biomarker Monitoring:** Home BP logging, lipid panel tracking (LDL-C, ApoB), glucose monitoring.
6. **Professional Care:** Structured dialogue guide for cardiologist / PCP consultation with PDF report.

# 6. Core Source Code Walkthrough

Below are the actual core PyTorch and FastAPI implementation modules driving the backend engine:

### A. PyTorch Deep Neural Network (DNN) — `backend/app/models/dnn_model.py`

```
import torch
import torch.nn as nn

class HeartDNN(nn.Module):
    """4-Layer Deep Neural Network with Batch Normalization & Dropout"""
    def __init__(self, input_dim: int = 13):
        super(HeartDNN, self).__init__()
        self.network = nn.Sequential(
            nn.Linear(input_dim, 64),
            nn.BatchNorm1d(64),
            nn.ReLU(),
            nn.Dropout(0.25),

            nn.Linear(64, 32),
            nn.BatchNorm1d(32),
            nn.ReLU(),
            nn.Dropout(0.20),

            nn.Linear(32, 16),
            nn.BatchNorm1d(16),
            nn.ReLU(),
            nn.Dropout(0.10),

            nn.Linear(16, 1),
            nn.Sigmoid()
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.network(x)

    def get_feature_saliency(self, x: torch.Tensor) -> torch.Tensor:
        """Compute input gradient saliency for Explainable AI"""
        x = x.clone().detach().requires_grad_(True)
        out = self.network(x)
        out.backward(torch.ones_like(out))
        return torch.abs(x.grad)
```

### B. TabNet Attentive Step Transformer — `backend/app/models/tabnet_model.py`

```
class TabNetStep(nn.Module):
    """Sequential Decision Step with Feature Transformer & Attentive Transformer"""
    def __init__(self, input_dim: int, feature_dim: int):
        super(TabNetStep, self).__init__()
        self.feature_transform = GLUBlock(input_dim, feature_dim)
        self.attentive_transform = nn.Sequential(
            nn.Linear(feature_dim, input_dim),
            nn.BatchNorm1d(input_dim)
        )

    def forward(self, x: torch.Tensor, prior_scales: torch.Tensor):
        # 1. Generate attention mask via sparse relaxation
        mask = torch.softmax(self.attentive_transform(prior_scales), dim=-1)
        # 2. Mask features and pass through Gated Linear Units (GLU)
        masked_x = x * mask
        decision_out = self.feature_transform(masked_x)
        return decision_out, mask
```

## C. Synchronized Inference & Ensemble Engine — `backend/app/ml/inference.py`

```
def predict_single(self, input_features: dict) -> dict:
    # 1. Transform raw biometrics using fitted StandardScaler (Zero Leakage)
    x_scaled = self.preprocessor.transform_single(input_features)
    x_tensor = torch.tensor(x_scaled, dtype=torch.float32)

    with torch.no_grad():
        p_dnn = float(self.dnn_model(x_tensor).squeeze().item())
        p_mlp = float(self.mlp_model(x_tensor).squeeze().item())
        p_tabnet, attention_masks = self.tabnet_model(x_tensor)
        p_tabnet = float(p_tabnet.squeeze().item())

    # 2. Performance-Weighted Calibrated Ensemble
    p_ensemble = (0.35 * p_dnn) + (0.30 * p_mlp) + (0.35 * p_tabnet)
    risk_percentage = round(p_ensemble * 100.0, 1)

    return {
        "dnn_prob": p_dnn, "mlp_prob": p_mlp, "tabnet_prob": p_tabnet,
        "ensemble_prob": p_ensemble, "risk_percentage": risk_percentage
    }
```

# 7. College Viva / Exam Questions & Model Answers

Top 15 most probable questions examiners and professors will ask about this project, along with high-scoring answers:

### Q1. What is the core objective of HeartGuard AI?

HeartGuard AI is an educational healthcare platform that estimates cardiovascular risk from 14 non-invasive clinical biomarkers using three machine learning architectures (DNN, MLP, TabNet). It delivers calibrated ensemble probabilities, Explainable AI (XAI) feature attributions, and evidence-based lifestyle recommendations without acting as a diagnostic system.

### Q2. Why did you choose three different models instead of just one?

In clinical tabular data, single models often suffer from architectural bias. Deep Neural Networks excel at complex non-linear feature interactions; Multi-Layer Perceptrons provide a regularized low-variance baseline; and TabNet provides decision-step attention interpretability. Combining them in a performance-weighted ensemble (35% DNN + 30% MLP + 35% TabNet) minimizes variance, maximizes generalization (0.934 AUC), and flags cases of model disagreement.

### Q3. How does TabNet differ from standard Multi-Layer Perceptrons?

Standard MLPs pass all features through fully connected layers simultaneously. TabNet uses sequential decision steps equipped with Attentive Transformers and Gated Linear Units (GLUs). At each step, TabNet applies a sparse attention mask to focus only on the most salient features for that specific patient, mimicking how cardiologists prioritize critical test findings.

### Q4. How do you prevent data leakage during preprocessing?

We enforce strict partitioning: 70% Train, 15% Validation, and 15% Test using Stratified Splitting. The StandardScaler (mean and variance) and median imputers are fitted ONLY on the 70% training set. The validation and test sets are only transformed using the saved training parameters, ensuring zero leakage of test distribution statistics.

### Q5. Why is ROC-AUC a more reliable metric than simple Accuracy in medical risk prediction?

In medical datasets, classification thresholds depend on clinical sensitivity requirements (minimizing False Negatives). Accuracy only measures correct predictions at a fixed 0.5 threshold. ROC-AUC evaluates the model's ability to rank risk across all possible decision thresholds, making it robust against class imbalance and threshold shifts.

### Q6. How does Explainable AI (XAI) work in this platform?

We extract feature attribution scores from two sources: (1) TabNet's sparse decision masks across attention steps, and (2) Neural gradient saliency (backpropagating output gradients to inputs). We then directionally classify each marker into 'Risk Elevators' (e.g. BP ≥ 140 mmHg) and 'Protective Factors' (e.g. Normal resting ECG).

### Q7. What is the purpose of Batch Normalization in your Deep Neural Network?

Batch Normalization standardizes the activations of intermediate layers during training. This mitigates internal covariate shift, enables higher learning rates, stabilizes gradient flow through deep layers, and provides a slight regularization effect.

### Q8. How does the system handle emergency cardiac symptoms?

The backend includes an emergency detection engine with compiled regex patterns for acute symptoms (e.g., crushing chest tightness, pain radiating to left arm/jaw, syncope, severe resting dyspnea). When triggered in the AI Assistant, it bypasses standard non-urgent advice and displays an immediate red emergency escalation alert with 911 / 112 guidance.

### Q9. What database and persistence strategy is implemented?

The backend implements a dual-mode persistence architecture: it connects to MongoDB when available (via PyMongo), and seamlessly falls back to a thread-safe, local JSON persistent store (`assessments_store.json`) when running offline or without an active database daemon.

### Q10. How is the frontend rendered and what makes it performant?

The frontend is built with React 18, TypeScript, and Vite. The real-time ECG rhythm monitor is rendered on an HTML5 `` using parametric P-Q-R-S-T cardiac waveform equations and double-buffering rather than heavy DOM manipulations, achieving a smooth 60 FPS.

### Q11. How does the backend communicate with the frontend in production?

The backend exposes RESTful endpoints with FastAPI and Pydantic v2 schemas. In production, the React frontend on Vercel communicates over HTTPS to the FastAPI server on Render, supported by dynamic CORS regex middleware (`https://.*\.vercel\.app`).

### Q12. What are the 6 Lifestyle Intelligence Pillars?

Nutrition (sodium & saturated fat limits, DASH diet), Physical Activity (150 mins aerobic exercise), Sleep Architecture (7-9 hours), Stress Management (HRV & vagal tone), Biomarker Monitoring (home BP & lipid logging), and Professional Care Collaboration.

### Q13. What is the clinical significance of the `oldpeak` and `thal` features?

`oldpeak` measures ST-segment depression in millimeters during stress testing, which indicates myocardial ischemia (oxygen deficit). `thal` represents nuclear perfusion imaging (thallium-201 scan) detecting normal blood flow, fixed scar defects (previous infarction), or reversible ischemic defects.

### Q14. How do you generate the downloadable PDF reports?

We use the Python ReportLab engine on FastAPI (`/api/v1/report/pdf/{id}`) to compile patient biometrics, risk scores, model breakdowns, XAI factor rankings, and lifestyle guidance into a formatted, downloadable PDF document.

### Q15. If given more time, what future enhancements would you implement?

1. Integrating 12-lead ECG time-series waveform classification using 1D CNNs or Transformers. 2. Longitudinal risk tracking over multiple patient visits. 3. HL7 / FHIR standard electronic health record (EHR) export interoperability.